Lab Assignment

Memory-Resident Database

std::list · std::map · std::multimap

Objective

In this lab you will build a memory-resident database of Person records using three cooperating C++ Standard Library containers:

- std::list<Person> — the primary data store; nodes never move, so pointers and iterators remain valid after insertions and deletions.
- std::map<string, iterator> — two secondary indexes (by ID and by Name) for O(log n) exact-match retrieval.
- std::multimap<string, iterator> — one secondary index (by Age) that tolerates duplicate keys.

Each index stores iterators into the list, not copies of the data. This is the core architectural idea: one copy of each record, many ways to find it.

Source Dataset

Use the Springfield People dataset from the course GitHub:
https://github.com/VMatosECC/Datasets/tree/main/Simpsons Download springfieldpeople.txt
and place it at c:/temp/springfieldpeople.txt

Background — Key Vocabulary

Term	Meaning
`std::list<T>`	Doubly-linked list; O(1) insert/delete anywhere; iterators survive insertions
`std::map<K,V>`	Balanced BST; unique keys; O(log n) find, insert, erase
`std::multimap<K,V>`	Like map but allows duplicate keys; equal_range() retrieves all matches
`iterator`	An object that points to a node in a container — like a pointer to one element
`prev(c.end())`	Returns an iterator to the last element (end() points past it)
`find_if(b,e,pred)`	Scans [b,e) and returns the first element for which pred returns true
`equal_range(key)`	Returns pair<iter,iter> — the range of all entries matching key
`lower_bound(key)`	Returns iterator to first entry >= key; used for prefix searches
`compare(pos,len,s)`	String method; faster prefix test than substr() — no temporary allocation
`runtime_error`	Standard exception for recoverable errors; thrown with a descriptive message

Architecture — One List, Three Indexes

Conceptual layout:

```
list<Person> db          ← primary store; every Person lives here exactly once

DBmap  idMap             ← key: id string      → iterator into db
DBmap  nameMap           ← key: name string    → iterator into db
DBmmap ageMultimap       ← key: age string     → iterator into db (duplicates OK)

After  db.push_back(p):
    idMap[p.id]          = prev(db.end());
    nameMap[p.name]      = prev(db.end());
    ageMultimap.emplace(p.age, prev(db.end()));
```

All three indexes point into the same list. Changing one Person record automatically affects every lookup — there is nothing to keep in sync.

Typedef vs using

The starter code uses **typedef map<string,list<Person>::iterator> DBmap**; to shorten the long type name.
In C++11 and later the equivalent is **using DBmap = map<string,list<Person>::iterator>;** — both are correct.
Either form is accepted in this lab.

Starter Files

File	Purpose
S0142-People-Database.cpp	Provided — contains experiment01() and experiment02()
Person.h	Provided — struct with id, name, age, info fields + operator<<
springfieldpeople.txt	Dataset — download from GitHub link above

Part 1: Study experiment01() — Build and Search

Open S0142-People-Database.cpp. Read experiment01() carefully before running anything. It demonstrates the full read-and-search cycle.

1.1 loadDatabase()

The chained getline loop reads four fields per line from the CSV:

```
while (getline(infile, p.id,   ',') &&
       getline(infile, p.name, ',') &&
       getline(infile, p.age,  ',') &&
       getline(infile, p.info) ) {
    db.push_back(p);
    idMap[p.id]   = prev(db.end());
    nameMap[p.name] = prev(db.end());
    ageMultimap.emplace(p.age, prev(db.end()));
}
```

Why chain the getline calls in the while condition?

If any field fails — truncated last line, file error — the entire while condition evaluates to false and the loop exits cleanly. A partial record is never added to the database.

1.2 exactSearch() — map lookup

The function calls m.find(key) which returns an iterator to the map entry, or m.end() if not found. *itm->second dereferences the list iterator stored in the map value, yielding the Person record.

```
auto itm = m.find(key);
if (itm == m.end())
    cout << "Not found\n";
else
    cout << "Found ==> " << *itm->second << "\n";
```

1.3 multiSearch() — multimap with equal_range

equal_range(key) returns a pair of iterators: [first, second) is the range of all entries sharing that key. Iterating it yields every Person with that age.

```
auto range = mm.equal_range(key);
if (range.first == range.second)
    cout << "No records found\n";
else
    for (auto it = range.first; it != range.second; ++it)
        cout << *it->second << "\n";
```

1.4 partialMatch() — lower_bound prefix search

lower_bound(key) finds the first map entry >= key in sorted order. Since all keys sharing a prefix are contiguous, walking forward until the prefix no longer matches covers all results. compare() is used instead of substr() to avoid creating a temporary string on every iteration.

```
auto it = m.lower_bound(key);
while (it != m.end() &&
       it->first.compare(0, key.size(), key) == 0) {
    cout << "Found ==> " << *it->second << "\n";
    ++it;
}
```

Checkpoint 1 — Run experiment01()

- Enable experiment01() in main(), comment out experiment02()
- Build and run. Verify showList prints all records correctly
- exactSearch by ID: search for 120 and 999 — one found, one not found
- exactSearch by NAME: search for a full name from the dataset
- multiSearch by AGE: search for an age shared by more than one person
- partialMatch: enter a single letter prefix — verify all matching names appear
- Enter q to exit each search loop

Part 2: Study experiment02() — Insert and Delete

experiment02() demonstrates full CRUD (Create-Read-Update-Delete): load → insert → verify → delete → verify. This part focuses on maintaining index consistency.

2.1 Duplicate guard before insert

```
if (idMap.count("200") == 1)
    throw runtime_error("Insert ignored - similar key already exists");
```

count() on a map returns 0 or 1 (keys are unique). Throwing runtime_error is the correct C++ idiom — see the try/catch in main() that catches it and prints a clean message instead of crashing.

2.2 Insert — three steps

```
db.push_back(newPerson);                // 1. add to the primary list
idMap[newPerson.id]   = prev(db.end()); // 2a. update map index
nameMap[newPerson.name] = prev(db.end());// 2b. update map index
ageMultimap.emplace(newPerson.age,       // 2c. update multimap index
                    prev(db.end()));
```

2.3 Delete — critical ordering

Always erase indexes BEFORE erasing the list node

Once db.erase(dbit) runs, the iterator dbit is invalid. Any index entry still referencing it becomes a dangling iterator — undefined behavior if accessed. Clean up all indexes first, then erase the list node.

```
// Step 1a: remove from map indexes (exact key match)
idMap.erase(dbit->id);
nameMap.erase(dbit->name);

// Step 1b: remove from multimap — erase ONLY the entry for this node
auto range = ageMultimap.equal_range(dbit->age);
for (auto ait = range.first; ait != range.second; ++ait) {
    if (ait->second == dbit) {    // same list node?
        ageMultimap.erase(ait);
        break;
    }
}

// Step 2: now safe to erase from the primary list
db.erase(dbit);
```

The multimap search is necessary because erasing by key (ageMultimap.erase(age)) would remove ALL records of that age, not just the one being deleted.

Checkpoint 2 — Run experiment02()

- Switch main() to call experiment02(), comment out experiment01()
- Wrap the call in try { experiment02(); } catch(const runtime_error& e) { cout << e.what(); }
- Run and confirm: record inserted, showIndices shows it in all three indexes
- Confirm: after delete, showList and showIndices no longer contain Otto Mann
- Manually change the insert ID to one that already exists in the file — confirm the duplicate guard throws and the message prints cleanly

Part 3: Lab Exercise — experiment03() Update Record

The starter file has a TODO placeholder for experiment03(). Implement it. An update modifies one or more fields of an existing record and keeps all three indexes consistent.

3.1 Strategy

Updating is more complex than insert or delete because the key itself may change (e.g. someone's name changes). The safe approach:

- Find the record using idMap (the ID never changes — treat it as the primary key).
- If the name or age field is changing, erase the old key from nameMap or ageMultimap before writing the new value.
- Write the new field values directly through the list iterator (dbit->name = newName; etc.).
- Re-insert the new key into nameMap or ageMultimap pointing to the same list node.
- The list iterator dbit remains valid — std::list does not invalidate iterators on modification.

3.2 Implement experiment03()

Your function must:

- Load the database (reuse loadDatabase()).
- Prompt the user for an ID to update.
- If not found, throw runtime_error with an appropriate message.
- Prompt for which field to update: name, age, or info.
- Apply the update to the list node and refresh the affected index.
- Call showList() and showIndices() after the update to verify correctness.

3.3 Test cases

Case	Expected behavior
`Update info only`	`No index change needed — info is not an index key`
`Update name`	`Must erase old nameMap entry, write new name, re-insert`
`Update age`	`Must erase old ageMultimap entry, write new age, re-insert`
`Update non-existent ID`	`Must throw runtime_error — no crash`

Checkpoint 3 — experiment03()

- Update an info field — showIndices unchanged, showList shows new info
- Update a name — nameMap shows the new name key; old key gone
- Update an age — ageMultimap shows new age key; old entry removed
- Attempt update of a missing ID — runtime_error message printed, program continues

Part 4: Challenge — Partial Match on the Age Multimap

partialMatch() is currently wired to nameMap (a map). Implement partialMatchMulti() that performs a prefix search on ageMultimap and returns all matching records.

For example, entering prefix `4` should return everyone whose age starts with 4: 40, 41, 42, 43, etc.

Starter signature:

```
void partialMatchMulti(const DBmmap& mm, string keyName = "") {
    string prefix;
    do {
        cout << "(" << keyName << ") Enter prefix [q to stop]: ";
        getline(cin, prefix);
        if (prefix == "q") break;

        // TODO: use lower_bound + compare loop on the multimap

    } while (true);
}
```

Hint

lower_bound and compare work identically on std::multimap as on std::map — the container is still sorted by key. Walk forward from lower_bound(prefix) while compare(0, prefix.size(), prefix) == 0.

Concept Questions

Answer each question in 2–4 sentences in your lab report or as comments in the code.

- Q1. Why does std::list guarantee that iterators stored in the index maps remain valid after inserting a new Person? Would std::vector provide the same guarantee?
- Q2. The code uses idMap[p.id] = prev(db.end()) to insert into the map. What happens if p.id already exists as a key? Is this a bug or intended behavior?
- Q3. Why must the indexes be updated BEFORE calling db.erase()? What goes wrong if the order is reversed?
- Q4. equal_range() is used in multiSearch() and in the delete logic. Explain what the returned pair represents and why a plain find() is not enough for the multimap.
- Q5. compare(0, key.size(), key) == 0 replaces substr(0, key.size()) == key in the partial match loop. What is the technical advantage of compare?

Self-Assessment Checklist

✓	I can …
☐	Explain why std::list is chosen as the primary store instead of std::vector or std::deque
☐	Declare typedef (or using) aliases for the map and multimap index types
☐	Write loadDatabase() using a chained getline while condition
☐	Explain why all three indexes are updated in the same loop iteration as db.push_back()
☐	Use map::find() and guard the result against .end() before dereferencing
☐	Use multimap::equal_range() to iterate all records matching a key
☐	Implement a prefix search with lower_bound() + compare() on a map
☐	Implement the same prefix search on a multimap
☐	Write a safe insert that checks for duplicates using count() before pushing
☐	Throw and catch runtime_error with a descriptive message
☐	Explain the mandatory ordering: erase all index entries BEFORE db.erase()
☐	Use equal_range + iterator comparison (ait->second == dbit) to erase exactly one multimap entry
☐	Write experiment03(): update a record and maintain all three indexes correctly
☐	Distinguish cases where an index must be refreshed (name/age change) from cases where it need not (info change)